

Supabase

Supabase 학습 체크리스트

- ☐ 백엔드 개발 없는 DB/스토리지 직접 제어 BaaS 아키텍처 원리 이해
- ☐ PostgreSQL 기반 RLS 정책 및 Anon Key의 유기적 보안 흐름 파악
- ☐ 테이블 생성에 따른 RESTful API 자동 빌드 및 Axios 데이터 조회 구현
- ☐ 공식 SDK 메서드 체이닝을 활용한 CRUD 및 단일 파일 업로드 구현

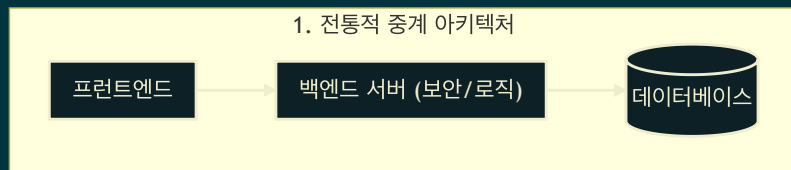
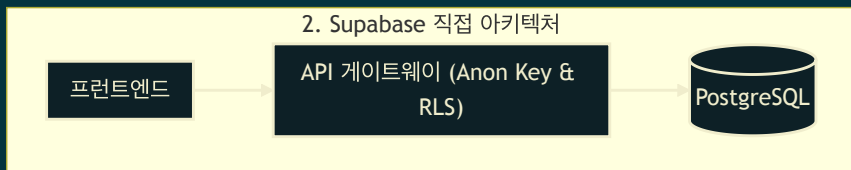
Supabase 개요

Supabase (Supabase)

PostgreSQL 데이터베이스 엔진 기반으로 구축되어 데이터베이스, 인증, 스토리지, 실시간 통신 등을 즉시 제공하는 오픈소스 BaaS 플랫폼

- **BaaS의 특징:** 백엔드 서버 인프라 구축 없는 즉각적인 고기능 클라우드 서비스 제공
- **핵심 기반:** 신뢰성 높은 PostgreSQL 엔진 기반으로 안정적인 관계형 데이터 관리 지원

아키텍처 비교: 전통적 구조 vs BaaS 구조



Supabase 보안 메커니즘 (Anon Key & RLS)

- Anon Key (익명 API 키)
 - 브라우저에 직접 노출되어도 무방한 클라이언트용 공개용 접근 식별자
- RLS (Row Level Security)
 - 테이블 단위 통제에서 벗어나 개별 행(Row) 수준으로 조회를 제한하는 핵심 기술
- RBAC (역할 기반 제어) 연동
 - 접속 유저 등급(`anon` 익명 유저, `authenticated` 로그인 유저 등)에 따른 투명한 데이터 필터링

데이터베이스 스키마와 RDB

스키마 (Schema)

데이터가 기록되는 표(Table)의 규격, 컬럼 데이터 타입 및 외래키 등 제약조건을 사전 정의한 데이터베이스 설계도

- **SQL**: 관계형 데이터베이스(RDB)를 제어하고 가공하는 질의 표준 언어
- **테이블 설계**: 데이터 간의 연계 관계를 논리적인 표 구조로 시각화하여 설계하는 과정

[실습] posts 테이블 생성 SQL

```
1  -- 게시글(posts) 테이블 정의
2  create table posts (
3      id bigint generated by default as identity primary key,
4      title text not null,
5      content text not null,
6      created_at timestamp with time zone default now() not null
7  );
```

[실습] comments 테이블 생성 SQL

```
1  -- 외래키 및 Cascade 삭제를 포함하는 댓글(comments) 테이블 정의
2  create table comments (
3      id bigint generated by default as identity primary key,
4      post_id bigint references posts(id) on delete cascade not null,
5      commenter text not null,
6      content text not null,
7      created_at timestamp with time zone default now() not null
8  );
```


[실습] profiles 테이블 생성 SQL

```
1  -- 사용자 프로필 정보 및 아바타 이미지 매핑용 테이블 정의
2  create table profiles (
3      id bigint generated by default as identity primary key,
4      nickname text not null,
5      avatar_url text,
6      updated_at timestamp with time zone default now() not null
7  );
```

Storage 버킷 신규 생성 및 Public 설정

- Storage Bucket 개념
 - 대용량 이미지, 문서 파일(바이너리)을 모아 보관하는 클라우드 가상 저장소
- Storage 설정 가이드
 - Supabase Storage 탭에서 신규 버킷 `avatars` 생성
 - **Public bucket 활성화 필수:** 별도의 중계 로직 없이 브라우저 즉시 렌더링 지원용

Supabase Auth & 인증 아키텍처

Supabase Auth (Auth)

클라이언트 단에서 직접 회원가입, 로그인, 세션 유지 등의 계정 관리를 수행할 수 있는 내장 보안 인증 서비스

- **JWT 기반 제어:** 가입 및 인증 성공 시 JSON Web Token을 로컬 스토리지에 자동 보관 및 관리
- **다양한 소셜 연동:** 동일한 메서드 구조 안에서 구글, 카카오 등 서드파티 OAuth 연동 즉각 확장

Auto-generated Data API 개요

- **엔드포인트 자동 빌드**
 - 별도의 API 라우트 개발 없이, DB 테이블 생성과 동시에 RESTful CRUD 통신용 API 즉각 가동
- **프론트엔드 직접 연동 환경 지원**
 - NPM 번들 프로세스 없이 즉시 사용 가능한 CDN 배포 스크립트 제공

```
1  <!-- Axios 및 Supabase JS SDK CDN 의존성 추가 -->
2  <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
3  <script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
```

REST API with Axios (데이터 조회)

```
1 // Axios 활용 REST API 헤더 및 파라미터 조립 조회 예시
2 const response = await axios.get(`${SUPABASE_URL}/rest/v1/posts`, {
3   headers: {
4     'apikey': SUPABASE_ANON_KEY,
5     'Authorization': `Bearer ${SUPABASE_ANON_KEY}`
6   },
7   params: { select: '*', order: 'created_at.desc' }
8 });
```

Supabase JS SDK 도입

- 클라이언트 환경 변수 선언
 - Supabase Project URL 및 Anon Key 상수로 정의
- SDK 자바스크립트 체이닝
 - 복잡한 API 주소 수동 조립 및 헤더 셋업을 배제하고 유연한 메서드 체이닝 구조 활용
- 초기화 및 SDK 가동
 - 단 1회의 클라이언트 인스턴스 생성으로 이후 모든 CRUD 및 파일 업로드 기능 총괄 처리

SDK 클라이언트 초기화 및 조회

```
1 // Supabase 클라이언트 초기화 및 생성
2 const supabaseClient = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
3
4 // 데이터 조회 (Select)
5 const { data, error } = await supabaseClient
6   .from('posts')
7   .select('*')
8   .order('created_at', { ascending: false });
```

SDK 데이터 등록 (Create)

```
1 // 데이터 등록 (Insert) 및 입력된 신규 행 반환
2 const { data, error } = await supabaseClient
3   .from('posts')
4   .insert([ { title, content, image_url: imageUrl } ])
5   .select();
```


Object Storage와 파일 업로드

- 바이너리 파일 저장 구조
 - 고용량 미디어 파일(이미지/비디오)은 DB가 아닌 Object Storage에 보관하고 URL 문자열만 DB에 기입
- SDK 연동 파일 업로드
 - 수동 파일 청크 전송 없이 SDK의 storage 메서드를 통하여 파일 바이너리를 클라우드에 직접 주입

SDK 기반 단일 파일 업로드

```
1  // 'avatars' 버킷 업로드 및 즉각적인 Public URL 획득
2  const { data, error } = await supabaseClient.storage
3    .from('avatars')
4    .upload(`public/${Date.now()}_${file.name}`, file);
5
6  if (error) throw error;
7
8  const { data: { publicUrl } } = supabaseClient.storage
9    .from('avatars')
10   .getPublicUrl(data.path);
```

프론트엔드 데이터 파이프라인

```
1 // 파일 스토리지 업로드 URL과 텍스트를 연계한 유기적 레코드 생성
2 async function handlePublish(title, content, file) {
3   const imageUrl = file ? await uploadFileWithSDK(file) : null;
4   const newPost = await createPostWithSDK(title, content, imageUrl);
5   console.log('DB 레코드 입력 성공:', newPost);
6 }
```